

BAB 6

ABSTRACT CLASS DAN INTERFACE

6.1 Pendahuluan

Pada BAB sebelumnya, mahasiswa telah mempelajari konsep **polymorphism**, yaitu kemampuan sebuah method untuk memiliki perilaku berbeda sesuai object yang menggunakannya. Konsep tersebut sangat penting karena membuat program lebih fleksibel dan mudah dikembangkan.

Pada BAB 6 ini, mahasiswa akan mempelajari dua konsep penting lain dalam OOP, yaitu **abstract class** dan **interface**. Keduanya digunakan untuk membuat rancangan class yang lebih terstruktur, terutama ketika beberapa class memiliki pola perilaku yang sama, tetapi cara menjalankannya berbeda.

Abstract class dan interface sering digunakan ketika programmer ingin membuat aturan dasar bagi class lain. Dengan konsep ini, program tidak hanya berjalan, tetapi juga memiliki desain yang lebih rapi dan mudah diperluas.

6.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep abstract class.
2. Menjelaskan konsep interface.
3. Membedakan abstract class dan interface.
4. Mengimplementasikan abstract class dalam Java.
5. Mengimplementasikan interface dalam Java.
6. Membuat program sederhana yang menggunakan abstract class dan interface.

6.3 Pengertian Abstract Class

Abstract class adalah class yang tidak dapat dibuat object secara langsung. Class ini digunakan sebagai class dasar atau template untuk class lain.

Abstract class biasanya berisi method yang masih bersifat umum. Sebagian method bisa sudah memiliki isi, sedangkan method lainnya belum memiliki isi dan harus diimplementasikan oleh subclass.

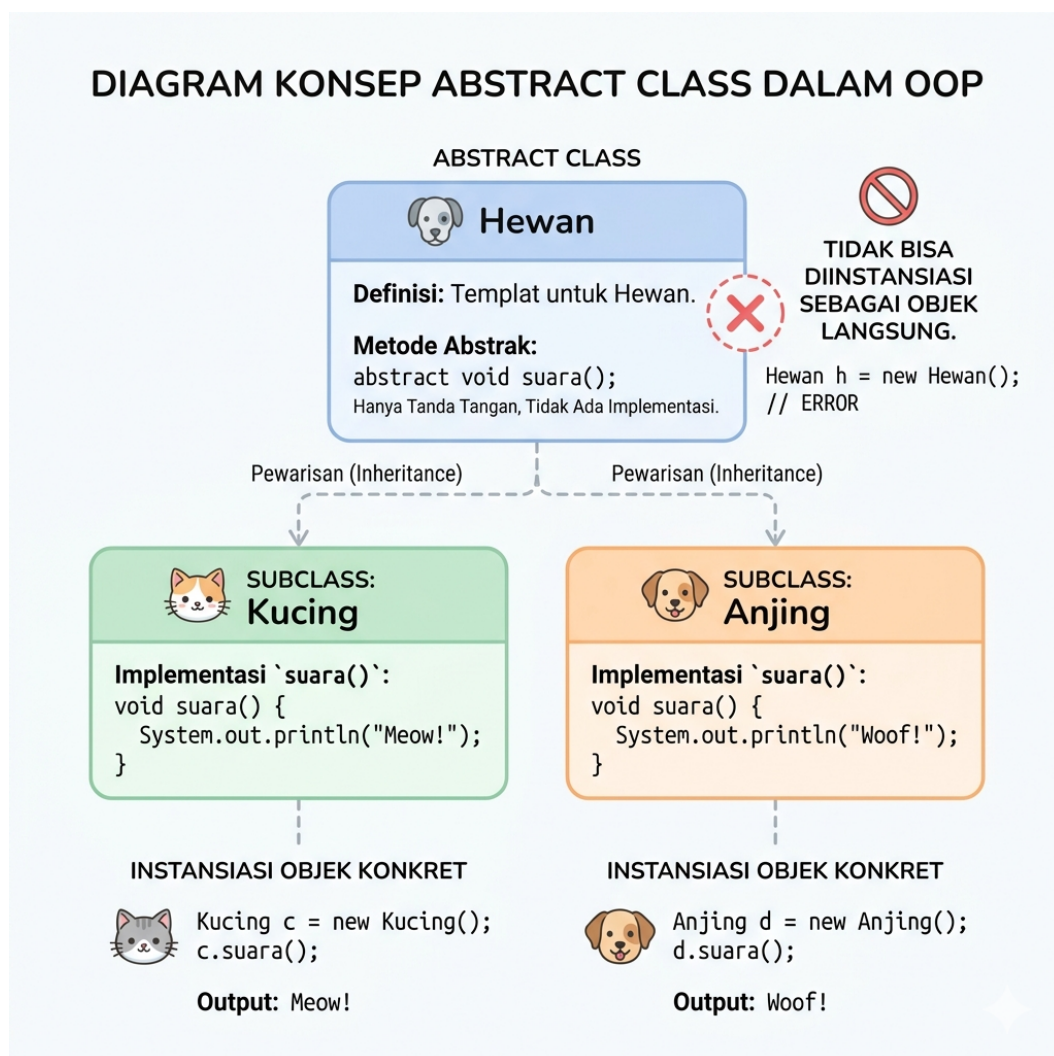
Contoh sederhana:

```
abstract class Hewan {  
    abstract void suara();  
  
    void makan() {  
        System.out.println("Hewan sedang makan");  
    }  
}
```

Pada contoh tersebut, class `Hewan` adalah abstract class. Method `suara()` belum memiliki isi, sehingga subclass wajib mengisi sendiri method tersebut.

6.4 Ilustrasi Abstract Class

Letakkan gambar pertama pada bagian ini, yaitu setelah subbab 6.3. Posisi ini tepat karena mahasiswa sudah membaca definisi abstract class, lalu langsung dibantu dengan visual.



Gambar 6.1 Diagram Konsep Abstract Class dalam OOP

Gambar ini menunjukkan bahwa abstract class **Hewan** berfungsi sebagai template bagi class turunannya. Class **Hewan** memiliki method abstract `suara()` yang belum memiliki implementasi, sehingga tidak dapat diinstansiasi langsung sebagai object. Class **Kucing** dan **Anjing** sebagai subclass harus mengimplementasikan method `suara()` sesuai perilaku masing-masing.

Dari gambar tersebut dapat dipahami bahwa abstract class berisi rancangan umum. Object konkret dibuat dari subclass, bukan dari abstract class langsung.

6.5 Ciri-Ciri Abstract Class

Abstract class memiliki beberapa ciri penting yang perlu dipahami.

1. Menggunakan keyword `abstract`.
2. Tidak bisa dibuat object secara langsung.
3. Dapat memiliki method abstract.
4. Dapat memiliki method biasa.
5. Dapat memiliki atribut dan constructor.
6. Subclass wajib mengimplementasikan method abstract, kecuali subclass tersebut juga abstract.

Contoh class yang tidak boleh diinstansiasi:

```
Hewan h = new Hewan(); // ERROR
```

Kode tersebut error karena `Hewan` adalah abstract class. Object harus dibuat dari subclass seperti `Kucing` atau `Anjing`.

6.6 Contoh Abstract Class dalam Java

Pada contoh berikut, class `Hewan` dibuat sebagai abstract class. Class `Kucing` dan `Anjing` menjadi subclass yang mengimplementasikan method `suara()`.

```
abstract class Hewan {
    String nama;

    Hewan(String nama) {
        this.nama = nama;
    }

    abstract void suara();

    void makan() {
        System.out.println(nama + " sedang makan.");
    }
}
```

```
class Kucing extends Hewan {  
  
    Kucing(String nama) {  
        super(nama);  
    }  
  
    @Override  
    void suara() {  
        System.out.println(nama + " mengeong: Meow!");  
    }  
}
```

```
class Anjing extends Hewan {  
  
    Anjing(String nama) {  
        super(nama);  
    }  
  
    @Override  
    void suara() {  
        System.out.println(nama + " menggonggong: Woof!");  
    }  
}
```

```
public class DemoAbstractClass {  
    public static void main(String[] args) {  
  
        Kucing k = new Kucing("Milo");  
        Anjing a = new Anjing("Buddy");  
  
        k.makan();  
        k.suara();  
  
        a.makan();  
        a.suara();  
    }  
}
```

Output:

```
Milo sedang makan.  
Milo mengeong: Meow!  
Buddy sedang makan.  
Buddy menggonggong: Woof!
```

Pada program tersebut, method `makan()` sudah tersedia di abstract class `Hewan`, sehingga dapat langsung digunakan oleh `Kucing` dan `Anjing`. Namun, method `suara()` harus dibuat ulang oleh masing-masing subclass karena method tersebut bersifat abstract.

6.7 Pengertian Interface

Interface adalah bentuk rancangan yang berisi daftar method yang harus dimiliki oleh class yang menggunakannya. Jika abstract class dapat dianggap sebagai template, maka interface dapat dianggap sebagai kontrak.

Interface hanya menentukan **apa yang harus dilakukan**, tetapi class yang mengimplementasikan interface menentukan **bagaimana cara melakukannya**.

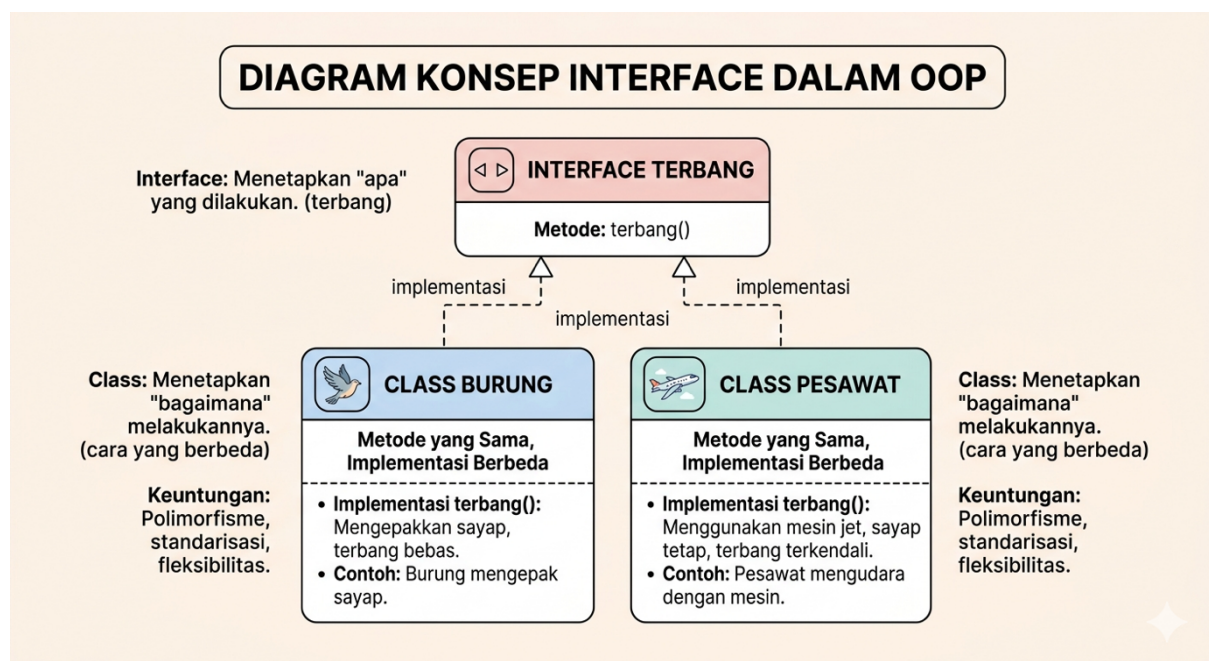
Contoh sederhana:

```
interface Terbang {  
    void terbang();  
}
```

Interface `Terbang` menyatakan bahwa setiap class yang mengimplementasikan interface ini harus memiliki method `terbang()`. Cara terbangnya bisa berbeda-beda sesuai class yang mengimplementasikannya.

6.8 Ilustrasi Interface

Letakkan gambar kedua pada bagian ini, yaitu setelah subbab 6.7. Posisi ini tepat karena mahasiswa sudah membaca definisi interface, lalu langsung melihat perbedaannya dengan class biasa.



Gambar 6.2 Diagram Konsep Interface dalam OOP

Gambar ini menunjukkan bahwa interface **Terbang** menetapkan method `terbang()` sebagai kemampuan yang harus dimiliki oleh class yang mengimplementasikannya. Class **Burung** dan **Pesawat** sama-sama mengimplementasikan interface tersebut, tetapi cara menjalankan method `terbang()` berbeda. Burung terbang dengan mengepakkan sayap, sedangkan pesawat terbang menggunakan mesin dan sayap tetap.

Dari gambar tersebut dapat dipahami bahwa interface berperan sebagai kontrak. Class yang menggunakan interface wajib menyediakan implementasi dari method yang ditentukan.

6.9 Ciri-Ciri Interface

Interface memiliki beberapa ciri utama.

1. Menggunakan keyword `interface`.
2. Diimplementasikan menggunakan keyword `implements`.
3. Tidak dapat dibuat object secara langsung.
4. Method dalam interface harus diimplementasikan oleh class yang menggunakannya.
5. Satu class dapat mengimplementasikan lebih dari satu interface.
6. Interface cocok digunakan untuk mendefinisikan kemampuan atau perilaku tertentu.

Contoh implementasi interface:

```
class Burung implements Terbang {  
    public void terbang() {  
        System.out.println("Burung terbang dengan mengepakkan sayap.");  
    }  
}
```

Pada contoh tersebut, class `Burung` wajib memiliki method `terbang()` karena class tersebut mengimplementasikan interface `Terbang`.

6.10 Contoh Interface dalam Java

Berikut contoh sederhana penggunaan interface `Terbang`.

```
interface Terbang {  
    void terbang();  
}
```

```
class Burung implements Terbang {  
  
    @Override  
    public void terbang() {  
        System.out.println("Burung terbang dengan mengepakkan sayap.");  
    }  
}
```

```
class Pesawat implements Terbang {  
  
    @Override  
    public void terbang() {  
        System.out.println("Pesawat terbang menggunakan mesin jet.");  
    }  
}
```

```

public class DemoInterface {
    public static void main(String[] args) {

        Burung b = new Burung();
        Pesawat p = new Pesawat();

        b.terbang();
        p.terbang();
    }
}

```

Output:

```

Burung terbang dengan mengepakkan sayap.
Pesawat terbang menggunakan mesin jet.

```

Pada contoh tersebut, `Burung` dan `Pesawat` sama-sama memiliki method `terbang()`. Namun, implementasinya berbeda karena cara burung dan pesawat terbang memang berbeda.

6.11 Perbedaan Abstract Class dan Interface

Abstract class dan interface sama-sama dapat digunakan untuk membuat rancangan program yang lebih rapi. Namun, keduanya memiliki perbedaan penting.

Aspek	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Cara menggunakan	<code>extends</code>	<code>implements</code>
Object langsung	Tidak bisa	Tidak bisa
Method biasa	Bisa memiliki method biasa	Umumnya berisi method yang harus diimplementasikan
Atribut	Bisa memiliki atribut biasa	Biasanya digunakan untuk konstanta
Constructor	Bisa memiliki constructor	Tidak memiliki constructor
Jumlah yang bisa digunakan	Satu class hanya bisa extends satu class	Satu class bisa implements banyak interface

Secara sederhana, gunakan abstract class jika beberapa class masih memiliki hubungan “adalah” atau **is-a**. Gunakan interface jika ingin menambahkan kemampuan tertentu yang bisa dimiliki oleh class yang berbeda jenis.

6.12 Kombinasi Abstract Class dan Interface

Dalam Java, sebuah class dapat mewarisi abstract class dan sekaligus mengimplementasikan interface. Ini sering digunakan ketika sebuah object memiliki identitas utama sekaligus kemampuan tambahan.

Misalnya, `Burung` adalah `Hewan`, maka dapat mewarisi abstract class `Hewan`. Namun, `Burung` juga memiliki kemampuan `Terbang`, maka dapat mengimplementasikan interface `Terbang`.

Contoh:

```
abstract class Hewan {
    String nama;

    Hewan(String nama) {
        this.nama = nama;
    }

    abstract void suara();

    void makan() {
        System.out.println(nama + " sedang makan.");
    }
}
```

```
interface Terbang {
    void terbang();
}
```

```
class Burung extends Hewan implements Terbang {

    Burung(String nama) {
        super(nama);
    }

    @Override
    void suara() {
        System.out.println(nama + " berkicau: Cuit... cuit...");
    }

    @Override
    public void terbang() {
        System.out.println(nama + " terbang dengan mengepakkan sayap.");
    }
}
```

```
public class DemoAbstractInterface {
    public static void main(String[] args) {

        Burung b = new Burung("Rio");

        b.makan();
        b.suara();
        b.terbang();
    }
}
```

Output:

```
Rio sedang makan.
Rio berkicau: Cuit... cuit...
Rio terbang dengan mengepakkan sayap.
```

Pada contoh tersebut, class `Burung` mewarisi abstract class `Hewan` dan mengimplementasikan interface `Terbang`. Dengan demikian, `Burung` wajib mengisi method `suara()` dari abstract class dan method `terbang()` dari interface.

6.13 Studi Kasus Praktikum: Data Hewan dan Kemampuan Terbang

Pada studi kasus ini, mahasiswa akan membuat program yang menggabungkan abstract class dan interface. Program dibuat dengan tema hewan agar mudah dipahami.

Ketentuannya:

- `Hewan` sebagai abstract class.
- `Terbang` sebagai interface.
- `Kucing` dan `Anjing` hanya mewarisi `Hewan`.
- `Burung` mewarisi `Hewan` dan mengimplementasikan `Terbang`.

Source Code

```
abstract class Hewan {
    String nama;
    int umur;

    Hewan(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    abstract void suara();

    void tampilInfo() {
        System.out.println("Nama : " + nama);
        System.out.println("Umur : " + umur + " tahun");
    }
}
```

```
interface Terbang {
    void terbang();
}
```

```
class Kucing extends Hewan {
    String ras;

    Kucing(String nama, int umur, String ras) {
        super(nama, umur);
        this.ras = ras;
    }

    @Override
    void suara() {
        System.out.println(nama + " mengeong: Meow!");
    }

    @Override
    void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jenis: Kucing");
        System.out.println("Ras : " + ras);
    }
}
```

```
class Anjing extends Hewan {
    String jenis;

    Anjing(String nama, int umur, String jenis) {
        super(nama, umur);
        this.jenis = jenis;
    }

    @Override
    void suara() {
        System.out.println(nama + " menggonggong: Woof!");
    }

    @Override
    void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jenis: Anjing");
        System.out.println("Tipe : " + jenis);
    }
}
```

```

class Burung extends Hewan implements Terbang {
    String warna;

    Burung(String nama, int umur, String warna) {
        super(nama, umur);
        this.warna = warna;
    }

    @Override
    void suara() {
        System.out.println(nama + " berkicau: Cuit... cuit...");
    }

    @Override
    public void terbang() {
        System.out.println(nama + " terbang dengan mengepakkan sayap.");
    }

    @Override
    void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jenis: Burung");
        System.out.println("Warna: " + warna);
    }
}

```

```

public class SistemHewan {
    public static void main(String[] args) {

        Hewan[] daftarHewan = {
            new Kucing("Milo", 2, "Persia"),
            new Anjing("Buddy", 3, "Golden Retriever"),
            new Burung("Rio", 1, "Hijau")
        };

        for (Hewan h : daftarHewan) {
            h.tampilInfo();
            h.suara();

            if (h instanceof Terbang) {
                Terbang t = (Terbang) h;
                t.terbang();
            }

            System.out.println("-----");
        }
    }
}

```

6.13.1

Output

```

Nama : Milo
Umur : 2 tahun
Jenis: Kucing
Ras  : Persia
Milo mengeong: Meow!
-----
Nama : Buddy
Umur : 3 tahun

```

```
Jenis: Anjing
Tipe : Golden Retriever
Buddy menggonggong: Woof!
-----
Nama : Rio
Umur : 1 tahun
Jenis: Burung
Warna: Hijau
Rio berkicau: Cuit... cuit...
Rio terbang dengan mengepakkan sayap.
-----
```

Pada studi kasus ini, array `daftarHewan` bertipe `Hewan`, sehingga dapat menyimpan object dari class `Kucing`, `Anjing`, dan `Burung`. Method `suara()` dijalankan sesuai implementasi masing-masing subclass.

Bagian `instanceof Terbang` digunakan untuk memeriksa apakah object memiliki kemampuan `Terbang`. Jika iya, maka object tersebut dapat dipanggil method `terbang()`.

6.14 Kesalahan Umum

Saat mempelajari abstract class dan interface, mahasiswa sering melakukan beberapa kesalahan. Kesalahan ini biasanya terjadi karena belum terbiasa membedakan class biasa, abstract class, dan interface.

Beberapa kesalahan umum adalah:

- 1. Membuat object langsung dari abstract class**
Contoh `new Hewan()` akan error jika `Hewan` adalah abstract class.
- 2. Tidak mengimplementasikan method abstract**
Subclass wajib mengisi method abstract, kecuali subclass tersebut juga dibuat abstract.
- 3. Salah menggunakan `extends` dan `implements`**
Abstract class digunakan dengan `extends`, sedangkan interface digunakan dengan `implements`.
- 4. Lupa menulis `public` pada method interface**
Method dari interface yang diimplementasikan pada class sebaiknya ditulis `public`.
- 5. Mengira interface sama dengan class biasa**
Interface lebih tepat dipahami sebagai kontrak perilaku, bukan sebagai object konkret.

6.15 Tugas Percobaan

Percobaan 1: Abstract Class

Buat abstract class `Hewan` dengan ketentuan berikut:

1. Memiliki atribut `nama`.
2. Memiliki constructor untuk mengisi `nama`.
3. Memiliki method abstract `suara()`.
4. Memiliki method biasa `makan()`.

Kemudian buat class `Kucing` dan `Anjing` yang mewarisi class `Hewan`. Setiap subclass harus mengimplementasikan method `suara()`.

Percobaan 2: Interface

Buat interface `Terbang` dengan method:

```
void terbang();
```

Kemudian buat class `Burung` dan `Pesawat` yang mengimplementasikan interface tersebut. Setiap class harus memiliki cara berbeda dalam menjalankan method `terbang()`.

Percobaan 3: Kombinasi Abstract Class dan Interface

Buat program dengan ketentuan berikut:

1. Abstract class `Hewan`.
2. Interface `Terbang`.
3. Class `Burung` mewarisi `Hewan` dan mengimplementasikan `Terbang`.
4. Class `Kucing` hanya mewarisi `Hewan`.
5. Tampilkan output dari method `suara()` dan `terbang()` jika object memiliki kemampuan terbang.

Percobaan 4: Studi Kasus Mini

Buat program **Sistem Data Hewan** dengan ketentuan berikut:

1. Memiliki abstract class `Hewan`.
2. Memiliki interface `Terbang`.
3. Memiliki minimal 3 subclass, misalnya `Kucing`, `Anjing`, dan `Burung`.
4. Setiap subclass mengimplementasikan method `suara()`.
5. Minimal satu subclass mengimplementasikan interface `Terbang`.
6. Gunakan array object untuk menyimpan minimal 5 object.
7. Tampilkan data, suara, dan kemampuan terbang jika ada.

6.16 Pertanyaan Diskusi

Jawablah pertanyaan berikut sebagai bagian dari pemahaman konsep.

1. Apa yang dimaksud dengan abstract class?
2. Mengapa abstract class tidak bisa dibuat object secara langsung?
3. Apa fungsi method abstract?
4. Apa yang dimaksud dengan interface?
5. Apa perbedaan `extends` dan `implements`?
6. Apa perbedaan abstract class dan interface?
7. Kapan sebaiknya menggunakan abstract class?
8. Kapan sebaiknya menggunakan interface?

6.17 Tugas Laporan Praktikum

Setelah menyelesaikan seluruh percobaan pada BAB 6, mahasiswa diminta membuat laporan praktikum. Format laporan dibuat konsisten dengan BAB sebelumnya agar mahasiswa terbiasa dengan pola kerja yang sistematis dan rapi.

Struktur laporan:

1. Cover
2. Tujuan praktikum
3. Alat dan bahan
4. Langkah percobaan
5. Source code
6. Hasil output program
7. Analisis program
8. Kesimpulan

Ketentuan isi analisis:

Pada bagian analisis, mahasiswa diminta menjelaskan penerapan abstract class dan interface pada program yang dibuat. Penjelasan minimal mencakup:

- class mana yang menjadi abstract class
- method mana yang bersifat abstract
- class mana yang menjadi subclass
- interface apa yang digunakan
- class mana yang mengimplementasikan interface
- perbedaan penggunaan `extends` dan `implements`
- perbedaan perilaku setiap subclass

Mahasiswa juga perlu menjelaskan mengapa abstract class atau interface digunakan dalam program tersebut. Penjelasan tidak perlu terlalu panjang, tetapi harus menunjukkan bahwa mahasiswa memahami konsep dasarnya.

Laporan dikumpulkan dalam bentuk **PDF** sesuai ketentuan dosen atau asisten praktikum.

6.18 Penutup

Pada BAB 6 ini, mahasiswa telah mempelajari abstract class dan interface sebagai konsep lanjutan dalam pemrograman berorientasi objek. Abstract class digunakan sebagai rancangan dasar yang masih dapat memiliki atribut, constructor, method biasa, dan method abstract.

Interface digunakan sebagai kontrak perilaku yang harus diimplementasikan oleh class tertentu. Dengan memahami abstract class dan interface, mahasiswa dapat membuat program yang lebih fleksibel, rapi, dan mudah dikembangkan.

Konsep ini sangat penting sebelum masuk ke materi relasi antar object pada bab berikutnya. Relasi antar object akan membantu mahasiswa memahami bagaimana beberapa class dapat saling bekerja sama dalam satu sistem.